

Лабораторная работа №12

дисциплина: Архитектура компьютера

Безходарнова Алиса Викторовна

Содержание

Цель работы	5
Задание	6
Выполнение лабораторной работы	7
Вывод	13
Контрольные вопросы	14
Список литературы	15

Список иллюстраций

1	Программа первая	7
2	Запуск	8
3	Вторая программа	8
4	Исполняемость файлов	9
5	Третья программа	10
6	Запуск	11
7	Четвертая программа	11
8	Запуск	12

Список таблиц

Цель работы

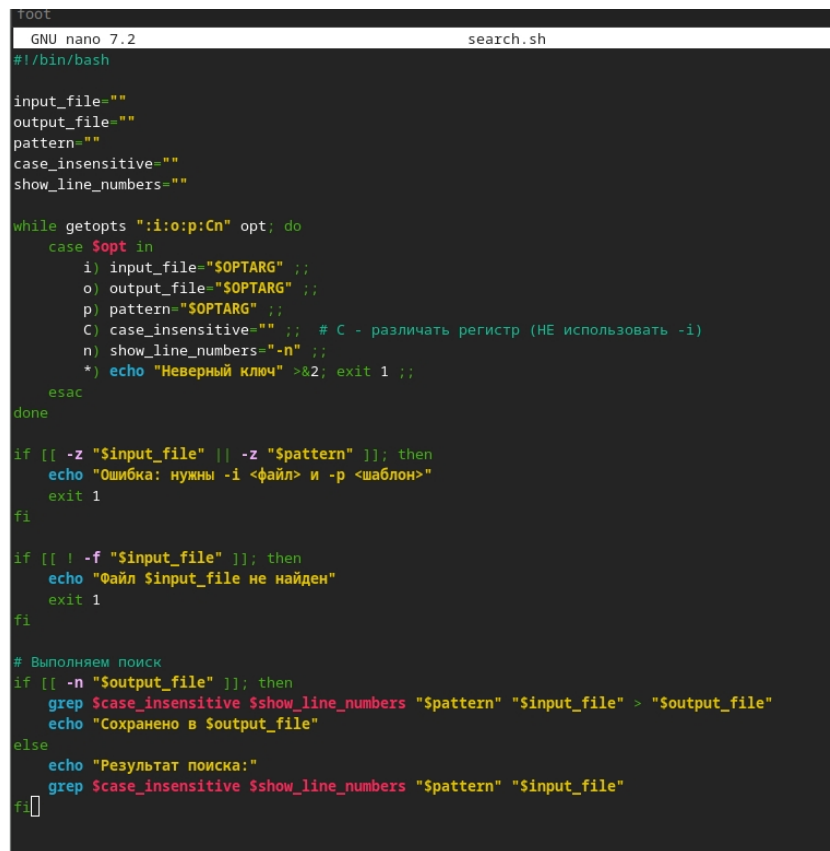
Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

Выполнение лабораторной работы

Создаю первую программу



```
foot
GNU nano 7.2 search.sh
#!/bin/bash

input_file=""
output_file=""
pattern=""
case_insensitive=""
show_line_numbers=""

while getopts ":i:o:p:Cn" opt; do
    case $opt in
        i) input_file="$OPTARG" ;;
        o) output_file="$OPTARG" ;;
        p) pattern="$OPTARG" ;;
        C) case_insensitive="" ;; # C - различать регистр (НЕ использовать -i)
        n) show_line_numbers="-n" ;;
        *) echo "Неверный ключ" >&2; exit 1 ;;
    esac
done

if [[ -z "$input_file" || -z "$pattern" ]]; then
    echo "Ошибка: нужны -i <файл> и -p <шаблон>"
    exit 1
fi

if [[ ! -f "$input_file" ]]; then
    echo "Файл $input_file не найден"
    exit 1
fi

# Выполняем поиск
if [[ -n "$output_file" ]]; then
    grep $case_insensitive $show_line_numbers "$pattern" "$input_file" > "$output_file"
    echo "Сохранено в $output_file"
else
    echo "Результат поиска:"
    grep $case_insensitive $show_line_numbers "$pattern" "$input_file"
fi
```

Рис. 1: Программа первая

Делаю ее исполняемой и запускаю (рис. @fig:002).

```

[avbezhodarnova1@fedora work]$ ./search.sh -i test.txt -p "hello"
Результат поиска:
[avbezhodarnova1@fedora work]$ ./search.sh -i test.txt -p "Hello"
Результат поиска:
[avbezhodarnova1@fedora work]$

```

Рис. 2: Запуск

Пишу вторую программу (Рис @fig:003).

```

foot
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;
    printf("Vvedite chislo: ");
    scanf("%d", &num);

    if (num > 0)
        exit(1);
    else if (num < 0)
        exit(2);
    else
        exit(0);
}

```

Рис. 3: Вторая программа

Делаю файл исполняемым и запускаю (Рис @fig:004)

```
vvedite chislo: 5
[avbezhodarnova1@fedora work]$ nano check_sign.sh
[avbezhodarnova1@fedora work]$ chmod +x check_sign.sh
./check_sign.sh
Запуск программы sign...
Vvedite chislo: 5

Результат: Число БОЛЬШЕ нуля
[avbezhodarnova1@fedora work]$
```

Рис. 4: Исполняемость файлов

Пишу третью программу (Рис @fig:005)

```
foot
GNU nano 7.2                                manage_files.sh
#!/bin/bash

# Проверка аргументов
if [ $# -ne 2 ]; then
    echo "Использование: $0 {create|delete} N"
    echo "Пример: $0 create 5"
    echo "           $0 delete 5"
    exit 1
fi

action=$1
n=$2

# Проверка, что N - число
if ! [[ $n =~ ^[0-9]+$ ]]; then
    echo "Ошибка: N должно быть положительным числом"
    exit 1
fi

case $action in
    create)
        echo "Создание $n файлов..."
        for i in $(seq 1 $n); do
            filename="${i}.tmp"
            touch "$filename"
            echo "  Создан: $filename"
        done
        echo "Готово! Создано $n файлов."
        ;;
    delete)
        echo "Удаление файлов..."
        deleted=0
        for i in $(seq 1 $n); do
            filename="${i}.tmp"
            if [ -f "$filename" ]; then
                rm "$filename"
                echo "  Удалён: $filename"
                ((deleted++))
            fi
        done
    esac
```

Рис. 5: Третья программа

И запускаю ее

```

[avbezhodarnova1@fedora work]$ nano manage_files.sh
[avbezhodarnova1@fedora work]$ chmod +x manage_files.sh
[avbezhodarnova1@fedora work]$ ./manage_files.sh create 5
Создание 5 файлов...
Создан: 1.tmp
Создан: 2.tmp
Создан: 3.tmp
Создан: 4.tmp
Создан: 5.tmp
Готово! Создано 5 файлов.
[avbezhodarnova1@fedora work]$ ls -la *.tmp
-rw-r--r--. 1 avbezhodarnova1 avbezhodarnova1 0 мая 9 13:34 1.tmp
-rw-r--r--. 1 avbezhodarnova1 avbezhodarnova1 0 мая 9 13:34 2.tmp
-rw-r--r--. 1 avbezhodarnova1 avbezhodarnova1 0 мая 9 13:34 3.tmp
-rw-r--r--. 1 avbezhodarnova1 avbezhodarnova1 0 мая 9 13:34 4.tmp
-rw-r--r--. 1 avbezhodarnova1 avbezhodarnova1 0 мая 9 13:34 5.tmp
[avbezhodarnova1@fedora work]$ ./manage_files.sh delete 5
Удаление файлов...
Удалён: 1.tmp
Удалён: 2.tmp
Удалён: 3.tmp
Удалён: 4.tmp
Удалён: 5.tmp
Готово! Удалено 5 файлов.
[avbezhodarnova1@fedora work]$

```

Рис. 6: Запуск

Пишу четвертую программу

```

GNU nano 2.9.2 archive_dir1.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <директория>"
    exit 1
fi

dir=$1

if [ ! -d "$dir" ]; then
    echo "Ошибка: директория $dir не существует"
    exit 1
fi

archive_name="archive_$(date +%Y%m%d_%H%M%S).tar.gz"
tar -czf "$archive_name" -C "$dir" .

echo "Архив создан: $archive_name"
echo "Размер архива: $(du -h $archive_name | cut -f1)"

```

Рис. 7: Четвертая программа

И запускаю ее

```
[avbezhodarno1@fedora work]$ nano archive_all.sh
[avbezhodarno1@fedora work]$ nano archive_recent.sh
[avbezhodarno1@fedora work]$ chmod +x archive_all.sh archive_recent.sh
[avbezhodarno1@fedora work]$ mkdir test_dir
[avbezhodarno1@fedora work]$ echo "old" > test_dir/old_file.txt
[avbezhodarno1@fedora work]$ touch -t 202001011200 test_dir/old_file.txt
[avbezhodarno1@fedora work]$ echo "new" > test_dir/new_file.txt
[avbezhodarno1@fedora work]$ ^C
[avbezhodarno1@fedora work]$ ./archive_all.sh test_dir
Архив создан: archive_20260509_133722.tar.gz
Размер архива: 4,0К
[avbezhodarno1@fedora work]$ ./archive_recent.sh test_dir
Поиск файлов, изменённых за последние 7 дней...
Найдено файлов: 1
Архив создан: recent_20260509_133729.tar.gz
Размер архива: 4,0К
[avbezhodarno1@fedora work]$
```

Рис. 8: Запуск

Вывод

В ходе данной лабораторной работы я изучила основы программирования в ОС Linux. Научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Контрольные вопросы

1. Каково предназначение команды `getopts`? Разбор аргументов командной строки в shell-скриптах.
2. Какое отношение метасимволы имеют к генерации имён файлов? Метасимволы (`*`, `?`, `[]`) используются для шаблонов при генерации списка файлов (wildcard expansion).
3. Какие операторы управления действиями вы знаете `if`, `case`, `for`, `while`, `until`, `select`.
4. Какие операторы используются для прерывания цикла? `break` (выход из цикла), `continue` (переход к следующей итерации).
5. Для чего нужны команды `false` и `true` Возвращают код завершения 1 (`false`) и 0 (`true`); используются в циклах и условных проверках.
6. Что означает строка `if test -f mans/i.\$s`, встреченная в командном файле? Проверяет, существует ли файл с именем, составленным из переменных `$s`, `$i` и точками, с экранированием специальных символов.
7. Объясните различие между конструкциями `while` и `until`. `while` выполняет цикл, пока условие истинно (код возврата 0); `until` выполняет цикл, пока условие ложно (код возврата не 0).

Список литературы